PROYECTO

El propósito del proyecto es diseñar e implementar una herramienta en **C++** que permita profundizar en el estudio de alguna de las **técnicas computacionales** abordadas en los capítulos relacionados con **Memoria y Discos**. La solución propuesta deberá cumplir con los **requerimientos establecidos**, permitiendo analizar y comparar el **rendimiento obtenido** en diferentes escenarios. Finalmente, los resultados deberán ser presentados de manera clara y sustentada, a través de un análisis comparativo detallado.

Alcance y Consideraciones

Para lograr este objetivo, cada equipo deberá implementar en **C++** uno de los proyectos listados a continuación. Es importante destacar que:

- La **implementación de un mismo proyecto** no podrá repetirse, lo que implica que un mismo proyecto no podrá ser desarrollado por más de un equipo. Al recibir este enunciado deben inmediatamente ponerse de acuerdo, seleccionar el proyecto y remitir un único documento con la relación de proyectos/equipos por la asignatura.
- Se valorará la **originalidad** en las propuestas. Si algún equipo tiene una idea de proyecto no contemplada en la lista, podrá **sugerirla para su evaluación y aprobación previa**.
- Los equipos de trabajo estarán conformados por **dos estudiantes**, quienes trabajarán en conjunto para llevar a cabo el desarrollo del proyecto. **Excepción:** Sólo podrán conformarse hasta **cuatro (4) equipos** de **tres (3) estudiantes,** integrados únicamente por los doce (12) estudiantes seleccionados durante clases debido al compromiso y desempeño demostrado.

LISTADO DE PROYECTOS

Simulación de Memoria Caché

1. **Simulador de Caché de Correspondencia Directa y asociativa por vías:** Implementar un caché de mapeo directo y asociativa por vías en C++ usando STL, que acepte una secuencia de direcciones y muestre estadísticas de aciertos y fallos. Comparar el rendimiento de diferentes esquemas de caché. Anticipar la carga de datos cercanos en tiempo o espacio antes de que se necesiten.
2. **Simulador de Caché Asociativa por Vías**: Implementar un caché de 2, 4 u 8 vías usando estructuras de datos como unordered_map o Boost Multi-Index. Implementar políticas de reemplazo como **LRU** o **FIFO**. Comparar el rendimiento de diferentes tamaños de caché. Anticipar la carga de datos cercanos en tiempo o espacio antes de que se necesiten.
3. **Simulador de Caché Asociativa por Vías Memoria Externa**: Implementar un caché asociativo de 2, 4 u 8 vías en C++ usando STXXL. Anticipar la carga de datos cercanos en tiempo o espacio antes de que se necesiten.
4. **Comparación de Políticas de Reemplazo (LRU, FIFO, LFU)**: Desarrollar un simulador de memoria caché que permita probar diferentes políticas de reemplazo y comparar su eficiencia. Medir el tiempo de ejecución y el hit ratio. Ajustar estructuras para gestionar eficientemente la política seleccionada.
5. **Implementación de un Caché Multinivel (L1, L2, L3)**: Diseñar un sistema que simule cómo las cachés en niveles interactúan entre sí y con la memoria principal. Modelar múltiples niveles de caché. Implementar políticas de coherencia entre niveles. Analizar el impacto del tamaño y latencia de cada nivel. Sincronización de datos entre niveles.

Optimización y Modelado de Caché

6. **Optimización de Bloques de Caché con Prefetching**: Implementar técnicas de prefetching para mejorar el rendimiento en accesos a memoria. Prefetch basado en patrones de acceso. Implementar prefetching secuencial y de salto. Analizar el impacto en la tasa de aciertos. Ajuste de la cantidad de bloques prefetch. Selección del Tamaño de Bloque.
7. **Simulación de Caché con STXXL para Grandes Volúmenes de Datos**: Usar STXXL para modelar una caché capaz de manejar grandes datasets en disco. Modelar la caché como un contenedor en disco. Se puede tomar en cuenta grandes datasets de áreas cómo sísmica o médicos. Utilizar contenedores **lineales** como vector y similares. Comparar al menos dos contenedores.
8. **Simulación de Caché con STXXL para Grandes Volúmenes de Datos**: Usar STXXL para modelar una caché capaz de manejar grandes datasets en disco. Modelar la caché como un contenedor en disco. Se puede tomar en cuenta grandes datasets de áreas cómo sísmica o médicos. Utilizar contenedores **jerárquicos** basados en árboles. Comparar al menos dos contenedores.
9. **Comparación de Boost Flat Map vs Google B-Tree en Caché**: Evaluar el rendimiento de estructuras de Boost y Google B-Tree en la administración de caché. Implementar cachés usando ambas estructuras. Medir rendimiento en búsqueda, inserción y eliminación. Comparar latencia y tasa de aciertos.
10. **Estrategias de Escritura en Caché (Write-Through vs Write-Back)**: Implementar y comparar estas dos estrategias de manejo de escritura en caché. Manejo de latencia y coherencia.
11. **Impacto de la Asociatividad en el Rendimiento de Caché**: Diseñar experimentos con distintas configuraciones de caché para evaluar la eficiencia en la tasa de aciertos. Implementar cachés con diferentes grados de asociatividad. Analizar tasa de aciertos y tiempo de acceso. Comparar resultados con cachés totalmente asociativas. Ajustar políticas de reemplazo en cachés con distintas asociatividades.

Memoria Virtual y Relación con Caché

12. **Simulador de Paginación con TLB** (Translation Lookaside Buffer) **y Caché**: Implementar una tabla de páginas y un TLB en conjunto con un simulador de caché. Medir tasa de aciertos del TLB y la caché. Evaluar impacto de fallos de página. Manejo de políticas de reemplazo tanto en TLB como en caché.
13. **Simulación de Memoria Virtual con LMDB**: Usar LMDB para gestionar grandes volúmenes de datos como si fuera memoria secundaria. Optimizar consultas y escritura. Medir tiempo de acceso y latencia. Manejo de grandes volúmenes de datos. Se puede tomar en cuenta grandes datasets de áreas sísmicas.
14. **Impacto del Tamaño de Línea de Caché en el Rendimiento**: Analizar cómo varía la tasa de aciertos al modificar el tamaño de línea en simulaciones. Simular diferentes tamaños de línea de caché. Medir impacto en tasa de aciertos y tiempo de acceso. Comparar resultados en distintos escenarios. Ajuste dinámico del tamaño de línea.

Integración con Aplicaciones Prácticas

15. **Análisis de Accesos a Memoria en Algoritmos de Ordenamiento**: Medir la cantidad de accesos a caché en algoritmos como QuickSort y MergeSort. Analizar impacto de caché en grandes datasets. Optimizar ordenamiento para minimizar fallos de caché. Ajuste de tamaño de bloque para minimizar E/S.

16. **Caché para Optimización de Simulación de Gráficos 3D**: Modelar un caché para mejorar el rendimiento en procesamiento gráfico. Patrón de acceso a datos. Selección de la Estructura de Datos y Técnica de Visualización.
17. **Simulador de Datos Sísmicos:** Procesar y analizar datos sísmicos almacenados en formato **SEGY** utilizando estructuras eficientes para la optimización de memoria y tiempos de acceso. Implementar un parser en **C++** para leer archivos SEGY (Se puede tomar alguno de los existentes). **STL Vector/Deque:** Para almacenar trazos leídos. **STXXL Vector/Map:** Para gestionar grandes volúmenes de datos en almacenamiento externo si el tamaño excede la memoria RAM. **Boost B-Tree/Flat Map:** Para indexar trazos por coordenadas o tiempo. Anticipar la carga de datos sísmicos cercanos en tiempo o espacio antes de que se necesiten.
18. **Simulador de Datos Médicos:** Procesar y analizar datos médicos utilizando estructuras eficientes para la optimización de memoria y tiempos de acceso. Implementar un parser en **C++** para leer los archivos (Se puede tomar alguno de los existentes). **Base de datos NoSQL** ligera para almacenamiento de metadatos. **STL ó STXXL Vector/Deque:** Para almacenar imágenes médicas en memoria. **Boost Multi-Index:** Para búsquedas rápidas de imágenes por atributos.

Discos

19. **Simulador de Arreglos RAID 0, 1, 5, 10 (Redundant Array of Independent Disks): RAID 0:** Distribución de datos sin redundancia. **RAID 1:** Espejo de datos para redundancia. **RAID 5:** Distribución de datos + paridad. **RAID 10:** Combinación de RAID 1 + 0. Modelar distribución de bloques de datos en discos. Simular fallos de discos y recuperación de datos. Simular fallos y reconstrucción de discos.
20. **Simulador de Algoritmos de Planificación en Discos:** Comparar el rendimiento de diferentes **algoritmos de planificación de discos**, analizando métricas clave como **tiempos de búsqueda, latencia promedio y tiempos de acceso**. Modelar un disco e implementar simulación de pistas y cilindros del disco. Simular movimientos del cabezal y tiempos de búsqueda/acceso. 1. **FCFS (First-Come, First-Served):** Atiende las solicitudes en el orden de llegada. 2. **SSTF (Shortest Seek Time First):** Atiende la solicitud más cercana al cabezal. 3. **SCAN (Elevator Algorithm):** Mueve el cabezal en una dirección atendiendo solicitudes y luego invierte. Implemente dos de estos algoritmos y Compare.
21. **Simulador de Caché de Correspondencia Directa y asociativa por vías en Sistemas de Archivos:** Implementar un caché de mapeo directo y asociativa por vías en C++ usando STL, que acepte una secuencia de direcciones y muestre estadísticas de aciertos y fallos, tomando en cuenta características de dos sistemas de archivos a comparar. Comparar el rendimiento de diferentes esquemas de caché para ambos sistemas de archivos.
22. **Simulador de Sistemas de Archivos con Bloques (FAT y Ext2):** Modelar cómo un sistema de archivos gestiona bloques en disco, simular operaciones de lectura/escritura y fragmentación. Representar un disco como un arreglo de bloques. Simular **estructuras de asignación de archivos**: **FAT (File Allocation Table):** tabla enlazada de bloques. **Ext2 (similar a Unix):** bloques directos, indirectos, dobles indirectos. Implementar operaciones básicas: Crear archivo (asignar bloques), Leer archivo (seguir los punteros), Eliminar archivo (liberar bloques).

El proyecto será desarrollado en **C++11 o superior** (C++14, C++17, C++20 o C++23), garantizando su correcta compilación y ejecución en un entorno **GNU/Linux** mediante el uso del **GNU Compiler Collection (GCC/G++)**. La implementación incluirá un archivo **Makefile** para automatizar el proceso de compilación y ejecución.

Según el proyecto seleccionado, se podrán utilizar; **Datos Sintéticos**: Generados artificialmente para modelar patrones controlados de acceso a memoria, como secuencias ordenadas, aleatorias o con comportamientos específicos; **Datos Aleatorios:** Conjuntos de datos creados a partir de distribuciones estadísticas (como uniforme, normal o exponencial), que simulan escenarios impredecibles y permiten analizar el comportamiento del simulador ante situaciones no determinísticas; **Datos Reales:** Trazas obtenidas de entornos reales, o extraídas de fuentes como Kaggle o Matrix Market o similares, que proporcionan un contexto práctico para evaluar la eficacia en condiciones reales de operación.

Infografías / Diapositivas como recurso de Ayuda

- **Repositorio.** En el repositorio de GitHub https://github.com/josedevlabs/computer-arch-labs/tree/main/docs/finals , se encuentran infografías o diapositivas realizadas con IA por estudiantes que ya cursaron la asignatura. Es importante puedan incluir la validación del recurso correspondiente en el desarrollo del proyecto seleccionado y su propuesta de mejora al recurso de ayuda: infografía/diapositiva con su respectivo prompt.

Elementos Requeridos para la Entrega

- **Repositorio.** Copia del repositorio en GitHub, incluyendo los conjuntos de datos de gran tamaño. Propuesta de Mejora al recurso de ayuda con su respectivo prompt. Una copia adicional del repositorio adjunta al correo de entrega, donde solo se debe incluir el código fuente. Se debe incluir informe (.tex y .pdf) y diapositivas.
- **Informe Técnico.** El informe deberá incluir las siguientes secciones: El Problema, Técnicas Computacionales para su Solución (algoritmos, estructuras de datos, etc.), Análisis y Comparación de Resultados.
- **Diapositivas:** Basadas en la estructura del informe.
- **Video:** Duración Aproximada de 15 a 30 minutos, cada integrante del equipo debe participar, se mostrarán y explicarán las diapositivas, así como la ejecución y secciones claves del proyecto, finalizando con Análisis y Discusión de los resultados. Subir a YouTube en Modo Oculto/No Listado. Si sube en modo privado no podrá ser visualizado. Se creará una playlist con los proyectos del curso.
- Se debe garantizar la correcta ejecución del proyecto y visualización de documentos (.tex y .pdf).
- La fecha tope para su recepción es viernes 16 de octubre de 2026 a las 11:59 pm.